ANALYSE COMPLÈTE — MODULE DÉPÔTS (Plateforme Bonzini)

Date : 2026-03-13

Auteur : Analyse automatique du code source

Objectif : Documenter exhaustivement le module dépôts pour servir de base au redesign UI

Version : v2.0 (post-migrations 2026-03)

TABLE DES MATIÈRES

1. [Schéma Base de Données](#1-schéma-base-de-données)
2. [Fonctions RPC](#2-fonctions-rpc)
3. [Cycle de Vie — Statuts et Transitions](#3-cycle-de-vie--statuts-et-transitions)
4. [Méthodes de Dépôt](#4-méthodes-de-dépôt)
5. [Gestion des Preuves](#5-gestion-des-preuves)
6. [Composants UI Admin](#6-composants-ui-admin)
7. [Composants UI Client](#7-composants-ui-client)
8. [Connexions avec Autres Modules](#8-connexions-avec-autres-modules)
9. [Problèmes et Incohérences Identifiés](#9-problèmes-et-incohérences-identifiés)
10. [Comparaison avec le Module Paiements](#10-comparaison-avec-le-module-paiements)
11. [Annexes](#11-annexes)

1. Schéma Base de Données

1.1 ENUMs

`deposit_status`

Créé dans 20251212074146, évolué par 3 migrations ultérieures :

Valeur	Migration d'ajout	Label UI	Couleur UI
created	20251212074146	Demande créée	Gris
awaiting_proof	20251212074146	En attente de preuve	Jaune
proof_submitted	20251212074146	Preuve envoyée	Bleu
admin_review	20251212074146	En vérification	Violet
validated	20251212074146	Validé	Vert
rejected	20251212074146	Rejeté	Rouge
pending_correction	20260131000001	À corriger	Orange
cancelled	20260219300000	Annulé	Gris

`deposit_method`

Créé dans 20251212074146 — immuable depuis :

Valeur DB	Label complet	Label court
bank_transfer	Virement bancaire	Virement
bank_cash	Dépôt cash banque	Cash banque
agency_cash	Cash agence Bonzini	Cash agence
om_transfer	Orange Money – Transfert	Orange UV
om_withdrawal	Orange Money – Retrait	Orange code
mtn_transfer	MTN MoMo – Transfert	MTN Float

mtn_withdrawal	MTN MoMo – Retrait	MTN code
wave	Wave	Wave

1.2 Table `deposits`

Migration de création : 20251212074146_3a54d3a0.sql
Colonnes ajoutées ultérieurement : 20260213000000_enhanced_deposit_validation.sql

```
CREATE TABLE public.deposits (
id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id         UUID REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,
reference       TEXT NOT NULL UNIQUE,                    -- ex: BZ-DP-2026-0042
amount_xaf      BIGINT NOT NULL CHECK (amount_xaf > 0),
method          deposit_method NOT NULL,
bank_name       TEXT,                                    -- si méthode bank_*
agency_name     TEXT,                                    -- si méthode agency_cash
client_phone    TEXT,                                    -- téléphone client déclaré
status          deposit_status NOT NULL DEFAULT 'created',
admin_comment   TEXT,                                    -- commentaire visible client
rejection_reason TEXT,                                   -- motif rejet/correction
validated_by    UUID REFERENCES auth.users(id),          -- admin qui a traité
validated_at    TIMESTAMPTZ,

-- Ajouts migration 20260213000000:
confirmed_amount_xaf  BIGINT,      -- montant réel si différent du déclaré
rejection_category    TEXT,        -- catégorie structurée du rejet
admin_internal_note   TEXT,        -- note interne (non visible client)

created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Contraintes :

- CHECK (amount_xaf > 0) — montant strictement positif
- UNIQUE (reference) — référence de type BZ-DP-YYYY-NNNN

Trigger :

```
CREATE TRIGGER update_deposits_updated_at
BEFORE UPDATE ON public.deposits
FOR EACH ROW EXECUTE FUNCTION public.update_updated_at_column();
```

Indexes :

- Index primaire sur id
- Index unique sur reference
- Index idx_performance_... ajouté en 20260301210000 (non listé dans ce fichier)

1.3 Table `deposit_proofs`

Migration de création : 20251212074146
Colonnes ajoutées : 20260213000000

```
CREATE TABLE public.deposit_proofs (
```

```
id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
deposit_id   UUID REFERENCES public.deposits(id) ON DELETE CASCADE NOT NULL,
file_url     TEXT NOT NULL,    -- chemin complet: 'deposit-proofs/{user_id}/{depositId}/...'
file_name    TEXT NOT NULL,
file_type    TEXT,

-- Colonnes initiales:
uploaded_at  TIMESTAMPTZ NOT NULL DEFAULT now(),

-- Ajouts 20260213000000:
uploaded_by       UUID REFERENCES auth.users(id),
uploaded_by_type  VARCHAR(10) DEFAULT 'client'
CHECK (uploaded_by_type IN ('client', 'admin')),
is_visible_to_client BOOLEAN DEFAULT TRUE,

-- Support soft-delete:
deleted_at   TIMESTAMPTZ,
deleted_by   UUID REFERENCES auth.users(id),
delete_reason TEXT
);
```

Indexes :

```
-- Index partiel sur les preuves actives (non supprimées)
CREATE INDEX idx_deposit_proofs_active
ON public.deposit_proofs(deposit_id) WHERE deleted_at IS NULL;
```

1.4 Table `deposit_timeline_events`

Migration de création : 20251212074146

```
CREATE TABLE public.deposit_timeline_events (
id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
deposit_id   UUID REFERENCES public.deposits(id) ON DELETE CASCADE NOT NULL,
event_type   TEXT NOT NULL,
description  TEXT NOT NULL,
performed_by UUID REFERENCES auth.users(id),
created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Types d'événements utilisés :

event_type	Déclenché par	Description
created	RPC create_client_deposit	Demande de dépôt créée
proof_submitted	Hook upload client/admin	Preuve envoyée
proof_added	Hook upload admin	Preuve(s) ajoutée(s) par l'admin
proof_deleted	Hook soft-delete	Preuve supprimée par admin/client
admin_review	RPC start_deposit_review	Vérification en cours
correction_requested	RPC request_deposit_correction	Correction demandée
resubmitted	RPC resubmit_deposit	Dépôt renvoyé après correction
validated	RPC validate_deposit	Dépôt validé
wallet_credited	RPC validate_deposit	Solde mis à jour avec +montant
rejected	RPC reject_deposit	Dépôt refusé
cancelled	RPC cancel_client_deposit	Dépôt annulé par le client

1.5 RLS Policies

Table `deposits`

Policy	Opération	Condition
Users can view own deposits	SELECT	auth.uid() = user_id
Users can create own deposits	INSERT	auth.uid() = user_id
Admins can view all deposits	SELECT	is_admin(auth.uid())
Admins can update deposits	UPDATE	is_admin(auth.uid())

Table `deposit_proofs`

Policy	Opération	Condition
Users can view own deposit proofs	SELECT	EXISTS dépôt de l'utilisateur
Admins can view all deposit proofs	SELECT	is_admin(auth.uid())
Users can upload proofs for own deposits	INSERT	EXISTS dépôt de l'utilisateur
Users can update own deposit proofs	UPDATE	EXISTS dépôt de l'utilisateur
Admins can update deposit proofs	UPDATE	is_admin(auth.uid())

Table `deposit_timeline_events`

Policy	Opération	Condition
Users can view own deposit timeline	SELECT	EXISTS dépôt de l'utilisateur
Admins can view all timelines	SELECT	is_admin(auth.uid())
System can insert timeline events	INSERT	true (permissif)

2. Fonctions RPC

2.1 `create_client_deposit` — Création d'un dépôt

Migration : 20260105122508_bd590dc0.sql

Appelé par : clients (via useCreateDeposit) ET admins (via useAdminCreateDeposit)

```
FUNCTION public.create_client_deposit(
p_user_id    UUID,
p_amount_xaf NUMERIC,
p_method     deposit_method,
p_bank_name  TEXT DEFAULT NULL,
p_agency_name TEXT DEFAULT NULL,
p_client_phone TEXT DEFAULT NULL
)
RETURNS JSON
```

Algorithme :

1. Génère une référence BZ-DP-YYYY-NNNN de manière atomique (lock table + retry jusqu'à 5 fois si collision)
2. Insère le dépôt avec status = 'created'
3. Insère un deposit_timeline_events de type created
4. Retourne { success: true, deposit_id, reference }

Retour erreur : { success: false, error: '...' }

Note : N'avance PAS automatiquement vers proof_submitted même si des preuves sont passées — l'upload est géré côté client après la création.

2.2 `validate_deposit` — Validation admin

Version active : 20260213000000_enhanced_deposit_validation.sql

```
FUNCTION public.validate_deposit(
p_deposit_id      UUID,
p_admin_comment   TEXT DEFAULT NULL,
p_confirmed_amount BIGINT DEFAULT NULL,  -- montant réel si différent du déclaré
p_send_notification BOOLEAN DEFAULT TRUE
)
RETURNS JSONB
```

Pré-requis :

- is_admin(auth.uid()) — réservé admins
- Statut "` validated et "` rejected
- Au moins 1 preuve active (deleted_at IS NULL) — OBLIGATOIRE

Algorithme :

1. Row lock FOR UPDATE sur le dépôt
2. Récupère le nom client depuis clients
3. Calcule le montant à créditer : v_credit_amount = COALESCE(p_confirmed_amount, amount_xaf)
4. Crée le wallet si inexistant (ON CONFLICT DO NOTHING)
5. Row lock FOR UPDATE sur le wallet
6. Calcule v_new_balance = wallet.balance_xaf + v_credit_amount
7. Met à jour wallets.balance_xaf
8. Met à jour deposits : status='validated', confirmed_amount_xaf (si différent du déclaré), validated_by, validated_at
9. Insère dans wallet_operations (legacy)
10. Insère dans ledger_entries type DEPOSIT_VALIDATED
11. Insère 2 événements timeline : validated + wallet_credited
12. Insère notification deposit_validated (si p_send_notification = true)
13. Insère admin_audit_logs
14. Retourne { success, amount_credited, old_balance, new_balance, reference }

2.3 `reject_deposit` — Rejet admin

Version active : 20260213000000 (overload étendu), fixé par 20260221500000

```
FUNCTION public.reject_deposit(
p_deposit_id         UUID,
p_reason             TEXT,                  -- OBLIGATOIRE
p_rejection_category TEXT DEFAULT NULL,     -- catégorie structurée
p_admin_note         TEXT DEFAULT NULL      -- note interne
)
RETURNS JSONB
```

Pré-requis :

- is_admin(auth.uid())
- Raison non vide

- Statut "` validated et "` rejected

Algorithme :

1. Row lock FOR UPDATE sur le dépôt
2. Met à jour deposits : status='rejected', rejection_reason, rejection_category, admin_internal_note, validated_by, validated_at
3. Insère dans ledger_entries type DEPOSIT_REFUSED (informationnel, balance_before = balance_after)
4. Insère événement timeline rejected
5. Insère notification deposit_rejected
6. Insère admin_audit_logs
7. Retourne { success, reference }

&þ Note : Le rejet NE débite PAS le wallet (le wallet n'est jamais crédité avant valida

---

2.4 `request_deposit_correction` — Demande de correction

Version active : 20260221500000 (fixe l'appel is_admin)

```
FUNCTION public.request_deposit_correction(
p_deposit_id UUID,
p_reason     TEXT   -- OBLIGATOIRE
)
RETURNS JSONB
```

Pré-requis :

- is_admin(auth.uid())
- Raison non vide
- Statut "` validated et "` rejected

Algorithme :

1. Met à jour deposits : status='pending_correction', rejection_reason=p_reason
2. Insère événement correction_requested
3. Insère notification deposit_correction_requested
4. Retourne { success, reference }

2.5 `start_deposit_review` — Prise en charge admin

Version active : 20260221500000

```
FUNCTION public.start_deposit_review(
p_deposit_id UUID
)
RETURNS JSONB
```

Pré-requis :

- is_admin(auth.uid())
- Statut DOIT être proof_submitted OU pending_correction

Algorithme :

1. Met à jour deposits : status='admin_review'
2. Insère événement admin_review

3. Retourne { success: true }

2.6 `resubmit_deposit` — Renvoi après correction (client)

Migration : 20260131200000

FUNCTION public.resubmit_deposit(
p_deposit_id UUID
)
RETURNS JSONB

Pré-requis :

- auth.uid() = deposit.user_id (ownership)
- Statut DOIT être pending_correction

Algorithme :

1. Met à jour deposits : status='proof_submitted', rejection_reason=NULL
2. Insère événement resubmitted
3. Retourne { success: true }

2.7 `cancel_client_deposit` — Annulation (client)

Migration : 20260219300000

FUNCTION public.cancel_client_deposit(
p_deposit_id UUID
)
RETURNS JSONB

Pré-requis :

- auth.uid() non NULL
- deposit.user_id = auth.uid() (ownership)
- Statut DOIT être dans ['created', 'awaiting_proof', 'proof_submitted']

Algorithme :

1. Row lock FOR UPDATE sur le dépôt
2. Met à jour deposits : status='cancelled'
3. Insère événement cancelled
4. Retourne { success: true, reference, message }

&þ Note : Aucune notification n'est envoyée à l'admin.

2.8 `get_deposit_stats` — Statistiques admin

Version active : 20260221500000

FUNCTION public.get_deposit_stats()
RETURNS JSONB

Retourne :
{
"total": 150,

"awaiting_proof": 5,
"proof_submitted": 12,
"pending_correction": 3,
"admin_review": 4,
"validated": 110,
"rejected": 8,
"to_process": 16,
"today_validated": 6,
"today_amount": 4500000
}

- to_process = proof_submitted + admin_review
- today_validated / today_amount = dépôts validés aujourd'hui (basé sur validated_at)

---

3. Cycle de Vie — Statuts et Transitions

3.1 Flux Standard (tous modes hors annulation)

```
[created]
%
%  upload preuve (client ou admin)
%¼
[proof_submitted] %Ä% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
%                                                  %
%  start_deposit_review                                  %
%¼                                                  %
[admin_review]                                                  %
%                                                  %
% % % % % % % % % % %<% % % % % % % % % % % % % % % % % % % % % %                                  %
%              %                          %                          %
%¼             %¼                         %¼                         %
[validated] [rejected]  [pending_correction]                  %
%                          %
%  resubmit_deposit      %
% % % % % % % % % % % % % % % % % % % % % % % % % %
```

3.2 Flux d'Annulation (client uniquement)

```
[created] % % %º [cancelled]
[awaiting_proof] % % %º [cancelled]
[proof_submitted] % % %º [cancelled]
```

3.3 Table Complète des Transitions

De	Vers	Déclencheur	Acteur	Condition
(création)	created	create_client_deposit	Client ou Admin	—
created	proof_submitted	Upload preuve	Client ou Admin	Statut IN uploadable states
awaiting_proof	proof_submitted	Upload preuve	Client ou Admin	—
pending_correction	proof_submitted	Upload preuve	Client ou Admin	—
proof_submitted	admin_review	start_deposit_review	Admin	Status = proof_submitted ou pending_correction
pending_correction	admin_review	start_deposit_review	Admin	(Accepté par RPC, mais pas par l'UI — voir §9.6) (RPC)

admin_review	validated	validate_deposit	Admin	Au moins 1 preuve active
admin_review	rejected	reject_deposit	Admin	Raison obligatoire
admin_review	pending_correction	request_deposit_correction	Admin	Raison obligatoire
proof_submitted	validated	validate_deposit	Admin	Au moins 1 preuve active
proof_submitted	rejected	reject_deposit	Admin	—
proof_submitted	pending_correction	request_deposit_correction	Admin	—
created	validated	validate_deposit	Admin	&þ Pas de guard statut (hors validated/rejected)
pending_correction	cancelled	cancel_client_deposit	Client	‘L Non autorisé (pas dans cancellable statuses)
created	cancelled	cancel_client_deposit	Client	—
proof_submitted	proof_submitted	resubmit_deposit	Client	Seulement depuis pending_correction
proof_submitted !’ created	created	Soft-delete preuve (aucune active restante)	Admin ou Client	Statut non-terminal

3.4 Statuts Terminaux

Les statuts validated, rejected, cancelled sont des états terminaux :

- isLocked = ['validated', 'rejected', 'cancelled'].includes(status) (composant détail)
- SLA ne s'applique plus (retourne null)
- Boutons d'action désactivés (sauf bouton supprimer pour super_admin)

3.5 Effets de Bord par Transition

Transition	Wallet	Ledger	Timeline	Notification
!’ created	aucun	aucun	created	—
!’ proof_submitted	aucun	aucun	proof_submitted ou proof_added	—
!’ admin_review	aucun	aucun	admin_review	—
!’ validated	+amount_xaf crédité	DEPOSIT_VALIDATED	validated + wallet_credited	deposit_validated
!’ rejected	aucun	DEPOSIT_REFUSED (info)	rejected	deposit_rejected
!’ pending_correction	aucun	aucun	correction_requested	deposit_correction_requested
!’ resubmitted	aucun	aucun	resubmitted	—
!’ cancelled	aucun	aucun	cancelled	—

4. Méthodes de Dépôt

4.1 Hiérarchie UI : Famille !’ Sous-méthode !’ DB

BANK (Banque) %%%% BANK_TRANSFER %%%º bank_transfer
%%%% BANK_CASH_DEPOSIT %%%º bank_cash

ORANGE_MONEY %%%%% OM_TRANSFER %%%º om_transfer
%%%%% OM_WITHDRAWAL %%%º om_withdrawal

MTN_MONEY %%%%%%% MTN_TRANSFER %%%º mtn_transfer
%%%%%%% MTN_WITHDRAWAL %%%º mtn_withdrawal

WAVE %%%%%%%%%%%% WAVE_TRANSFER %%%º wave
(pas de sous-méthode distincte)

AGENCY_BONZINI %% AGENCY_CASH %%%º agency_cash
(pas de choix de sous-méthode)

4.2 Champs Additionnels par Méthode

Méthode	bank_name	agency_name	Données supplémentaires
bank_transfer	Requis (banque)	—	IBAN, SWIFT, N° Compte (affichés dans le récapitulatif)
bank_cash	Requis (banque)	—	N° Compte, Titulaire (affichés)

agency_cash	—	Requis (agence)	Adresse, horaires (affichés)
om_transfer	—	—	N° Orange Money (affiché), instructions USSD
om_withdrawal	—	—	Code Marchand généré (#150*...#)
mtn_transfer	—	—	N° MTN Float (affiché), instructions
mtn_withdrawal	—	—	Code Marchand généré
wave	—	—	N° Wave (affiché), instructions app

4.3 Banques Supportées

Code	Label
ECOBANK	Ecobank
CCA	CCA Bank
UBA	UBA
AFRILAND	Afriland First Bank
OTHER	Autre banque

4.4 Agences Bonzini

Code	Label	Adresse
DOUALA_BONAPRISO	Douala – Bonapriso	(défini dans depositMethodsData.ts)
DOUALA_BONAMOUSSADI	Douala – Bonamoussadi	(défini dans depositMethodsData.ts)
YAOUNDE_CENTRE	Yaoundé – Centre	(défini dans depositMethodsData.ts)

4.5 Timeline Method Family

Le composant timeline adapte ses libellés selon la méthode :

Famille timeline	Méthodes	Libellé étape 1	Libellé étape 2
standard	bank_transfer, bank_cash, om_transfer, mtn_transfer, wave	"Dépôt déclaré"	"Preuve envoyée"
withdrawal	om_withdrawal, mtn_withdrawal	"Retrait déclaré"	"Code fourni"
agency	agency_cash	"Dépôt en agence"	"Reçu confirmé"

4.6 Limite Mobile Money

Un avertissement s'affiche dans MobileNewDeposit si le montant dépasse MOBILE_MONEY_TRANSACTION_LIMIT (constante définie dans depositMethodsData.ts). Les méthodes bancaires ou Wave ne sont pas concernées.

---

5. Gestion des Preuves

5.1 Bucket Storage Supabase

Bucket : deposit-proofs

Path pattern : {user_id}/{depositId}/{timestamp}-{random}.{ext}

Durée URL signée : 3600 secondes (1h)

Exemple de chemin stocké :

deposit-proofs/550e8400-e29b-41d4-a716-446655440000/3fa85f64.../1741872000000-abc123.jpg

5.2 Upload — Client

Hook : useUploadProof (1 fichier) / useUploadMultipleProofs (N fichiers)

Source : src/hooks/useDeposits.ts

Flux :

1. validateUploadFile(rawFile) — validation taille/type
2. compressImage(rawFile) — compression images

3. Upload vers bucket deposit-proofs
4. Insert dans deposit_proofs : uploaded_by_type='client'
5. Lit le statut courant du dépôt
6. Si statut IN ['created', 'awaiting_proof', 'pending_correction'] !' UPDATE deposits
7. Insert deposit_timeline_events de type proof_submitted

Conditions d'upload côté UI : non-locked (statut "` validated/rejected/cancelled)

5.3 Upload — Admin

Hook : useAdminUploadProofs
Source : src/hooks/useAdminDeposits.ts

Flux :

1. compressImage(rawFile) — compression (&þ validateUploadFile NON appelé ici)
2. Upload vers bucket deposit-proofs
3. Insert dans deposit_proofs : uploaded_by_type='admin', uploaded_by=admin.id
4. Si statut IN ['created', 'awaiting_proof', 'pending_correction'] !' UPDATE deposits
5. Insert deposit_timeline_events de type proof_added

Conditions d'upload côté UI : canAddProof = !isLocked

5.4 Upload à la Création Admin (MobileNewDeposit)

Dans useAdminCreateDeposit :

1. RPC create_client_deposit !' obtient deposit_id
2. Si proofFiles fournis !' upload jusqu'à 5 fichiers
3. Si au moins 1 upload réussi !' UPDATE deposits SET status='proof_submitted'
4. Insert deposit_timeline_events de type proof_submitted
5. Insert admin_audit_logs : create_deposit_for_client

5.5 Soft-Delete — Admin

Hook : useAdminDeleteProof

Flux :

1. UPDATE deposit_proofs SET deleted_at, deleted_by, delete_reason (soft-delete)
2. Compte les preuves actives restantes (deleted_at IS NULL)
3. Si 0 preuves restantes ET statut "` terminal !' UPDATE deposits SET status='create
4. Insert deposit_timeline_events de type proof_deleted
5. Insert admin_audit_logs : delete_deposit_proof

5.6 Soft-Delete — Client

Hook : useDeleteDepositProof

Flux :

1. UPDATE deposit_proofs SET deleted_at, deleted_by, delete_reason (soft-delete)
2. Si 0 preuves actives restantes ET statut "` terminal !' UPDATE deposits SET status
3. Insert deposit_timeline_events de type proof_deleted

5.7 Guard de Validation

La RPC validate_deposit vérifie obligatoirement :

```
SELECT COUNT(*) FROM deposit_proofs
WHERE deposit_id = p_deposit_id AND deleted_at IS NULL;
-- Si COUNT = 0 !' RETURN error 'Aucune preuve - impossible de valider'
```

5.8 Raisons de Suppression Prédéfinies

PROOF_DELETE_REASONS = ['Upload incorrect', 'Mauvais dépôt', 'Doublon', 'Image illisible', 'Autre']

5.9 Affichage dans la Galerie (MobileDepositDetail)

Chaque thumb de preuve affiche :

- Image si file_type.startsWith('image/') + URL signée disponible
- Icône document sinon (PDF)
- Badge Admin ou Client (en haut à gauche)
- Badge date dd/MM HH:mm (en bas à gauche)
- Bouton "Voir" (full screen) si URL signée disponible
- Bouton "Télécharger" (lien download) si URL signée disponible
- Bouton "Supprimer" (rouge) si !isLocked

---

6. Composants UI Admin

6.1 `MobileDepositDetail.tsx`

Fichier : src/mobile/screens/deposits/MobileDepositDetail.tsx

Taille : ~1400 lignes

Route : /m/deposits/:depositId

6.1.1 Données chargées

Hook	Données	Cache
useAdminDepositDetail	Dépôt + profil client	staleTime: 10s
useAdminDepositProofs	Preuves + signed URLs	staleTime: 55min
useAdminDepositTimeline	Événements timeline	staleTime: default
useAdminWalletByUserId	Solde actuel du client	—

6.1.2 Variables Computées

```
isLocked       = ['validated', 'rejected', 'cancelled'].includes(status)
canValidate    = !isLocked
canReject      = !isLocked
canStartReview = status === 'proof_submitted'    // &þ exclut pending_correction
canAddProof    = !isLocked
isSuperAdmin   = currentUser?.role === 'super_admin'
hasProofs      = proofs && proofs.length > 0
slaLevel       = getDepositSlaLevel(created_at, status)  // fresh/aging/overdue/null
amountDiffers  = confirmedAmount !== deposit.amount_xaf && confirmedAmount > 0
```

6.1.3 Sections de la Page

1. Status Banner — gradient de couleur selon statut, badge statut, icône lock si terminal, SLA dot, date relative
2. Amount Hero Card — montant déclaré, montant confirmé (si différent + barré), méthode, banque/agence, bouton "Télécharger le relevé" (PDF)
3. Client Info Card — avatar initiales, nom, téléphone, company, solde actuel wallet /m/clients/{user_id}

4. Preuves — titre + compteur, galerie horizontale (scroll), warning si aucune preuve + non-locked, bouton "Ajouter"

5. Détails du dépôt (expandable) — référence (mono), méthode, banque, agence, date création, commentaire admin, motif rejet + catégorie, note interne admin

6. Timeline — 4 steps (ou terminal), icônes état, dates formatées, connecteurs

7. Bouton Supprimer — visible si (!isLocked || isSuperAdmin)

6.1.4 Action Bar (sticky, bottom)

Visible si canValidate || canReject || canStartReview :

Bouton	Couleur	Condition d'affichage
"Commencer la vérification"	Violet	canStartReview (status = proof_submitted)
"Rejeter"	Rouge border	canValidate (!isLocked)
"Corriger"	Orange border	canValidate (!isLocked)
"Valider"	Vert fond	canValidate (!isLocked)

Layout : "Commencer la vérification" seul en haut si applicable, puis ligne "Rejeter + Corriger" (50%/50%), puis "Valider" pleine largeur.

6.1.5 Bottom Sheets (modales)

Sheet de Validation :

- Montant déclaré (lecture seule)
- Input "Montant confirmé" (XAF, éditable — si différent = montant réel crédité)
- Textarea "Commentaire admin" (visible client)
- Toggle "Envoyer une notification" (défaut = ON)
- Bouton "Confirmer la validation"

Sheet de Rejet :

- Sélection catégorie (dropdown REJECTION_REASONS)
- Textarea "Message pour le client"
- Textarea "Note interne admin" (non visible client)
- Bouton "Confirmer le rejet" (désactivé si catégorie ou message vide)

Sheet de Correction :

- Textarea raison (free-text)
- Bouton "Envoyer la demande de correction"

Sheet d'Upload :

- Input file (accepte images + PDF)
- Liste des fichiers sélectionnés
- Bouton "Uploader les preuves"

Sheet Suppression Preuve :

- Sélection raison (prédéfinie ou "Autre")
- Si "Autre" !' textarea texte libre
- Bouton "Confirmer la suppression"

Sheet Suppression Dépôt :

- Message de confirmation
- Bouton rouge "Supprimer définitivement"

6.1.6 Raisons de Rejet Prédéfinies (`REJECTION_REASONS`)

['Montant incorrect', 'Preuve illisible', 'Référence absente', 'Mauvais compte bancaire', 'Document non conforme', 'Suspicion / incohérence', 'Autre']

6.2 `MobileDepositsScreen.tsx`

Fichier : src/mobile/screens/deposits/MobileDepositsScreen.tsx

Taille : 517 lignes

Route : /m/deposits

6.2.1 KPI Cards (scroll horizontal)

Card	Couleur	Source stat	Cliquable !' filtre
À traiter	Bleu	stats.to_process	statusFilter = 'to_process'
À corriger	Orange	stats.pending_correction	statusFilter = 'pending_correction'
Validés	Vert	stats.validated	statusFilter = 'validated'
Aujourd'hui	Primary (si > 0)	stats.today_validated + today_amount	Non cliquable

6.2.2 Filtres Disponibles

Chips statut (toujours visibles) :

Filtre	Statuts SQL mappés
Tous	(aucun filtre)
À traiter	proof_submitted + admin_review
À corriger	pending_correction
Validés	validated
Rejetés	rejected
Annulés	cancelled

Filtres avancés (panel toggle) :

- Méthode : Toutes méthodes + 8 méthodes individuelles (chips)
- Tri : Plus récent !" (défaut), Plus ancien !', Montant !", Montant !' (chips)
- Période : Date de (input date) !' Date à (input date)

Recherche libre (barre) :

- Debounced
- Appliquée côté client sur les items déjà chargés
- Champs recherchés : first_name + last_name, reference, phone

6.2.3 Liste des Dépôts

Pagination : Infinite scroll (taille de page = QUERY_LIMITS.ITEMS_PER_PAGE)

Pull-to-refresh : Oui

Chaque ligne affiche :

- Avatar initiales (2 lettres) + Nom complet
- Référence + méthode + count preuves (icône trombone)
- Montant (droite) + badge statut (couleur) + SLA dot + date relative

SLA dot :

- Ø=ßâ fresh : < 2h (sla-fresh)
- Ø=ßá aging : 2-8h (sla-aging)
- Ø=Ý4 overdue : > 8h (sla-overdue, animé)
- Absent pour les statuts terminaux

6.3 `MobileNewDeposit.tsx`

Fichier : src/mobile/screens/deposits/new-deposit/MobileNewDeposit.tsx

Taille : 1001 lignes

Route : /m/deposits/new ou /m/deposits/new?clientId={id}

6.3.1 Steps et Navigation

Steps disponibles: client | amount | family | submethod | bank | agency | recap | creating

Step	Affiché si	Contenu
client	Pas de ?clientId	Recherche + liste clients
amount	Toujours	Input montant + presets
family	Toujours	Sélection famille méthode
submethod	Famille avec plusieurs sous-méthodes (BANK, OM, MTN)	Sélection sous-méthode
bank	Sous-méthode BANK_TRANSFER ou BANK_CASH_DEPOSIT	Sélection banque
agency	Famille AGENCY_BONZINI	Sélection agence
recap	Toujours	Récapitulatif + confirmation
creating	Pendant création	Loader

Navigation automatique :

- AGENCY_BONZINI !' skip submethod !' agency
- WAVE !' skip submethod !' recap
- BANK !' submethod !' si bank_transfer ou bank_cash !' bank !' recap
- ORANGE_MONEY / MTN_MONEY !' submethod !' recap (pas de bank step)

Progress bar 3 phases :

- Phase 0 : Informations (steps client!'agency)
- Phase 1 : Récapitulatif
- Phase 2 : Création

6.3.2 Détail de Chaque Step

Step client :

- Input recherche (nom / téléphone)
- Liste max 20 résultats
- Clic !' sélection + passage à amount
- Pré-sélection via URL ?clientId=xxx

Step amount :

- Input numérique (filtre non-chiffres)
- Affichage formaté formatXAF avec animation useCountUp
- Boutons presets : 100K, 500K, 1M, 2M
- Bouton "Continuer" désactivé si amount < 1000

Step family :

- 5 cartes cliquables (icône + label + description)
- Avertissement si amountNum > MOBILE_MONEY_TRANSACTION_LIMIT

Step submethod :

- Liste des sous-méthodes pour la famille sélectionnée

Step bank :

- 5 cartes banques (Ecobank, CCA, UBA, Afriland, Autre)

Step agency :

- 3 cartes agences (adresse + horaires)

Step recap :

- Card récapitulatif : montant + méthode
- Card client
- Card "Coordonnées de dépôt" : champs copiables (icône copy !' check), code march
- Card "Instructions" : liste numérotée
- Section "Preuves (optionnel)" : upload max 5 fichiers (images ou PDFs)
- Section "Commentaire admin (optionnel)" : textarea
- Notice de confirmation
- Bouton "Confirmer et créer le dépôt"

Step creating :

- Loader animé (animate-deposit-pulse)

6.3.3 Logique de Création

```
await createDeposit.mutateAsync({
user_id: selectedClient.user_id,
amount_xaf: amountNum,
method: getDepositMethod(),  // mapping SubMethod !' DepositMethod
bank_name: ...,
agency_name: ...,
admin_comment: adminComment || undefined,
proofFiles: proofFiles.length > 0 ? proofFiles : undefined,
});
// !' navigate(/m/deposits/${result.id})
```

---

7. Composants UI Client

7.1 Pages Client

Fichier	Route	Description
src/pages/DepositsPage.tsx	/deposits	Liste des dépôts du client
src/pages/NewDepositPage.tsx	/deposits/new	Création dépôt (4 étapes)
src/pages/DepositDetailPage.tsx	/deposits/:id	Détail d'un dépôt client

7.2 Hooks Client

Hook	Source	Description
useMyDeposits	useDeposits.ts	Liste des dépôts du client (50 derniers)
useDepositDetail	useDeposits.ts	Détail d'un dépôt
useDepositProofs	useDeposits.ts	Preuves avec signed URLs
useDepositTimeline	useDeposits.ts	Événements timeline
useCreateDeposit	useDeposits.ts	Création via RPC
useUploadProof	useDeposits.ts	Upload 1 fichier
useUploadMultipleProofs	useDeposits.ts	Upload multiple
useDeleteDepositProof	useDeposits.ts	Soft-delete preuve
useCancelDeposit	useDeposits.ts	Annulation dépôt

useResubmitDeposit	useDeposits.ts	Renvoi après correction

7.3 Différences Client vs Admin

Aspect	Client	Admin
Client Supabase	supabase (client-auth)	supabaseAdmin (admin-auth)
Peut créer	Oui (son propre dépôt)	Oui (pour n'importe quel client)
Peut valider	'L	'
Peut rejeter	'L	'
Peut demander correction	'L	'
Peut annuler	' (si statut cancellable)	'L (supprime au lieu)
Peut réenvoyer	' (depuis pending_correction)	'L
Peut supprimer preuve	' (soft-delete)	' (soft-delete + audit)
Peut télécharger PDF reçu	' (depuis DepositDetailPage)	' (depuis MobileDepositDetail)

---

8. Connexions avec Autres Modules

8.1 Wallet

- Crédité à : validation uniquement (RPC validate_deposit)
- Montant crédité : COALESCE(confirmed_amount, declared_amount)
- Guard : wallets.balance_xaf >= 0 (CHECK constraint — ne peut pas aller négatif)
- Jamais débité dans le module dépôts (contrairement aux paiements)

8.2 Ledger Entries

Type d'entrée	Déclenché par	Effet balance
DEPOSIT_VALIDATED	validate_deposit	+amount_xaf !' wallet crédité
DEPOSIT_REFUSED	reject_deposit	balance_before = balance_after (informatif)

8.3 Wallet Operations (legacy)

validate_deposit crée toujours une entrée wallet_operations de type 'deposit'. Cette table est normalement légacy (remplacée par ledger_entries) mais reste peuplée pour les dépôts.

```
&þ Si wallet_operations a été droppée par la migration 20260221300000_drop_legacy_tables.sq
(capturé par le bloc EXCEPTION WHEN OTHERS).
```

8.4 Notifications

Type	Déclenché par	Message
deposit_validated	validate_deposit	"Votre dépôt de X XAF a été validé. Nouveau solde: Y XAF"
deposit_rejected	reject_deposit	"Votre dépôt de X XAF a été refusé. Motif: ..."
deposit_correction_requested	request_deposit_correction	"Veuillez corriger votre dépôt REF. Motif: ..."

8.5 Dashboard

- queryClient.invalidateQueries(['dashboard-stats']) déclenché après :
- Validation d'un dépôt
- Création d'un dépôt (admin)
- get_dashboard_stats (RPC séparé) inclut les stats de dépôts

8.6 Fiche Client

- MobileDepositDetail !' bouton lien vers /m/clients/{user_id} (ExternalLink)

- MobileNewDeposit !' accessible depuis fiche client via ?clientId=xxx
- L'historique des dépôts du client est probablement affiché dans la fiche client

8.7 PDF (Reçu de Dépôt)

- Template : src/lib/pdf/templates/DepositReceiptPDF.tsx
- Téléchargement : downloadPDF() (utilise @react-pdf/renderer)
- Données passées : DepositReceiptData (id, reference, dates, amounts, method, bank/agency, client info)
- Nom de fichier : recu_depot_{reference}_{clientName}.pdf

8.8 Audit Logs

Toutes les actions admin sont tracées dans admin_audit_logs :

Action	Données trackées
validate_deposit	reference, client, declared/confirmed amount, old/new balance, notification sent
reject_deposit	reference, client, amount, category, reason, admin note
delete_deposit_proof	deposit_id, proof_id, reason
create_deposit_for_client	client_id, amount, method, proof count

---

9. Problèmes et Incohérences Identifiés

9.1 &þ Double Écriture `wallet_operations` Probablement Cassée

Problème : La fonction validate_deposit (v20260213) insère toujours dans wallet_operations (table legacy) ET dans ledger_entries. La migration 20260221300000_drop_legacy_tables.sql a possiblement droppé wallet_operations.

Impact : Si la table est droppée, le bloc EXCEPTION WHEN OTHERS de la RPC capturera l'erreur silencieusement et retournera { success: false, error: SQLERRM }. La validation échouera.

À vérifier : Regarder si wallet_operations existe encore en base. Si non, retirer l'INSERT legacy de validate_deposit.

9.2 &þ Statut `awaiting_proof` — Fantôme Non Utilisable

Problème : L'ENUM deposit_status contient awaiting_proof (migration initiale 20251212074146), mais :

- Aucune RPC ne crée un dépôt avec ce statut
- Aucune transition n'y amène
- STATUS_INDEX['awaiting_proof'] = 0 (traité comme created dans la timeline)
- get_deposit_stats le regroupe avec created dans awaiting_proof
- Il est dans UPLOADABLE_STATES / v_cancellable_statuses (protections redondantes)

Impact : Confusion pour les développeurs. Si un dépôt se retrouve dans cet état (import manuel, migration ancienne), il ne se distingue pas de created à l'écran.

Recommandation : Soit l'utiliser intentionnellement (créer une transition vers ce statut quand le client tarde à uploader), soit le déprécier formellement.

9.3 &þ `canStartReview` Trop Restrictif — Incohérence UI/RPC

Problème : Dans MobileDepositDetail.tsx :

const canStartReview = deposit.status === 'proof_submitted';

Mais la RPC start_deposit_review accepte aussi pending_correction comme statut source.

Impact : Un dépôt en pending_correction (correction demandée mais pas encore resoumise par le client) ne peut pas passer en admin_review depuis l'UI admin. L'admin est bloqué.

Scénario problématique : Admin demande une correction, puis change d'avis et veut r
impossible via l'UI.

9.4 &þ `useDeleteDeposit` — Suppression Directe Sans RPC

Problème : Le hook useDeleteDeposit fait des DELETE directs sur deposit_proofs, deposit_timeline_events, et deposits via supabaseAdmin sans passer par un RPC sécurisé.

Impact :

- Pas de vérification de statut côté DB (un admin normal pourrait appeler l'API directement et supprimer un dépôt validated)
- Pas d'entrée dans admin_audit_logs
- L'UI protège via (!isLocked || isSuperAdmin) mais c'est une protection côté client uniquement

Risque : Suppression accidentelle d'un dépôt validé si la logique UI est contournée.

9.5 &þ `useAdminUploadProofs` — Pas de `validateUploadFile`

Problème : Dans useAdminCreateDeposit, validateUploadFile est appelé. Dans useAdminUploadProofs (upload depuis le détail), il ne l'est pas.

Impact : Un admin peut uploader des fichiers de grande taille ou de types non autorisés depuis la page de détail, mais pas depuis la création.

9.6 &þ Race Condition sur Avancement de Statut à l'Upload

Problème : L'avancement de statut !' proof_submitted lors d'un upload se fait via une s
TypeScript :

1. Upload Storage
2. Insert deposit_proofs
3. SELECT status
4. IF uploadable !' UPDATE status

Si deux admins uploadent simultanément, les deux lisent le même statut et mettent à
possibles dans la timeline.

Recommandation : Encapsuler dans un RPC transactionnel submit_deposit_proof.

9.7 &þ `cancel_client_deposit` — Pas de Notification Admin

Problème : Quand un client annule son dépôt, aucune notification n'est envoyée à l'équipe admin. Si un admin était en train de préparer la validation, il peut ne pas voir immédiatement le changement de statut.

Impact : Faible si l'UI se met à jour en temps réel, mais pas de push notification admin.

9.8 &þ `validate_deposit` Appelle `is_admin(v_admin_id)` mais `rej
aussi — Cohérence Partielle

Constaté : La migration 20260221500000_fix_deposit_rpcs_is_admin.sql corrige les 4 RPCs qui appelaient is_admin() sans paramètre. La version en 20260213000000 utilise déjà is_admin(v_admin_id). Il y a eu une période d'incohérence entre ces migrations, résolue.

État actuel : Cohérent. Toutes les RPCs appellent is_admin(v_admin_id) correctement.

9.9 &þ Confirmed Amount Non Vérifié Côté SQL

Problème : validate_deposit accepte n'importe quelle valeur pour p_confirmed_amount sans vérifier que c'est > 0 ou dans une plage raisonnable par rapport au montant déclaré.

Scénario : Un admin peut confirmer un montant de 1 XAF pour un dépôt de 10M XAF !'

Recommandation : Ajouter une vérification p_confirmed_amount > 0 et optionnellement un garde sur l'écart max par rapport au montant déclaré.

9.10 !9þ Statut `awaiting_proof` Visible dans `get_deposit_stats`

get_deposit_stats inclut :

'awaiting_proof', COUNT(*) FILTER (WHERE status IN ('created', 'awaiting_proof'))

Cette agrégation retourne awaiting_proof = (created + awaiting_proof). Trompeuse car on s'attend à un comptage du statut spécifique.

---

10. Comparaison avec le Module Paiements

10.1 Différences Architecturales

Aspect	Module Dépôts	Module Paiements
Wallet	Crédité à la validation	Débité à la création
Statuts	8 statuts (dont pending_correction, cancelled)	8 statuts (dont cash_pending, cash_scanned)
Preuves	Upload client + admin, soft-delete	Upload client + admin, RPC delete
Création	Client ou admin, RPC create_client_deposit	Client ou admin, RPC create_payment
Bénéficiaire	Pas de bénéficiaire	Bénéficiaire requis (table beneficiaries)
Taux de change	Non applicable	daily_rates + calcul CNY
Cash	agency_cash (Bonzini agency)	cash method avec QR code + signature
Correction	pending_correction + resubmit	N/A
Annulation	Client peut annuler	N/A
Ledger validation	DEPOSIT_VALIDATED	PAYMENT_EXECUTED
Ledger rejet	DEPOSIT_REFUSED	PAYMENT_CANCELLED_REFUNDED

10.2 Similitudes

- Même stack : React + TanStack Query + Supabase + TypeScript
- Même pattern hooks : useAdminXxx (supabaseAdmin) vs useXxx (supabase client)
- Même soft-delete sur les preuves
- Même audit logs admin_audit_logs
- Même système de notifications
- Même PDF receipt (@react-pdf/renderer)
- Même infinite scroll (usePaginatedXxx)
- Même pattern SLA (getDepositSlaLevel vs getPaymentSlaLevel)

10.3 Ce qui Existe dans Paiements mais Pas dans Dépôts

- Bénéficiaires sauvegardés (table beneficiaries)
- Flow cash avec QR code + signature

- Rate is custom (taux forcé)
- Commentaire visible client distinct du commentaire admin
- Restriction d'annulation plus granulaire par rôle

10.4 Ce qui Existe dans Dépôts mais Pas dans Paiements

- Statut pending_correction + workflow de correction/resoumission
- Montant confirmé différent du montant déclaré
- Catégorie de rejet structurée
- Note interne admin séparée
- Annulation par le client
- Familles de méthodes + sous-méthodes (hiérarchie UI)
- SLA indicator dans la liste (absent de la liste paiements)

---

11. Annexes

11.1 Liste des Fichiers Critiques

Fichier	Rôle
src/hooks/useDeposits.ts	Hooks client (create, upload, cancel, resubmit, delete proof)
src/hooks/useAdminDeposits.ts	Hooks admin (validate, reject, correct, review, upload, delete)
src/hooks/usePaginatedDeposits.ts	Pagination infinie pour la liste admin
src/lib/depositTimeline.ts	Construction des steps de timeline, SLA
src/types/deposit.ts	Types TS, labels, couleurs, constantes
src/data/depositMethodsData.ts	Données méthodes (banques, agences, infos comptes)
src/mobile/screens/deposits/MobileDepositDetail.tsx	Fiche dépôt admin (détail + actions)
src/mobile/screens/deposits/MobileDepositsScreen.tsx	Liste dépôts admin (filtres, KPIs, liste)
src/mobile/screens/deposits/new-deposit/MobileNewDeposit.tsx	Création dépôt admin (multi-step)
src/pages/DepositDetailPage.tsx	Fiche dépôt client (web)
src/pages/NewDepositPage.tsx	Création dépôt client (web)
src/pages/DepositsPage.tsx	Liste dépôts client (web)
src/lib/pdf/templates/DepositReceiptPDF.tsx	Template PDF reçu dépôt
supabase/migrations/20251212074146_*.sql	Schema initial (tables, ENUMs, RPCs v1)
supabase/migrations/20260105122508_*.sql	RPC create_client_deposit atomique
supabase/migrations/20260131000001_*.sql	Ajout statut pending_correction
supabase/migrations/20260131200000_*.sql	RPCs validate/reject/correction/review v2
supabase/migrations/20260213000000_*.sql	Enhanced validation (confirmed amount, soft-delete proofs)
supabase/migrations/20260214000000_*.sql	RLS policies deposit_proofs
supabase/migrations/20260219300000_*.sql	Suppression rate limit + ajout cancelled + RPC cancel
supabase/migrations/20260221500000_*.sql	Fix is_admin() dans toutes les RPCs dépôts

---

11.2 Schéma SQL Complet — Table `deposits` (État Final)

```
CREATE TABLE public.deposits (
id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id         UUID REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,
reference       TEXT NOT NULL UNIQUE,
amount_xaf      BIGINT NOT NULL CHECK (amount_xaf > 0),
method          deposit_method NOT NULL,
bank_name       TEXT,
```

```
agency_name          TEXT,
client_phone         TEXT,
status               deposit_status NOT NULL DEFAULT 'created',
admin_comment        TEXT,
rejection_reason     TEXT,
validated_by         UUID REFERENCES auth.users(id),
validated_at         TIMESTAMPTZ,
-- Ajouts 20260213000000:
confirmed_amount_xaf BIGINT,
rejection_category   TEXT,
admin_internal_note  TEXT,
-- Timestamps:
created_at           TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at           TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

11.3 Schéma SQL Complet — Table `deposit_proofs` (État Final)

```
CREATE TABLE public.deposit_proofs (
id                   UUID PRIMARY KEY DEFAULT gen_random_uuid(),
deposit_id           UUID REFERENCES public.deposits(id) ON DELETE CASCADE NOT NULL,
file_url             TEXT NOT NULL,
file_name            TEXT NOT NULL,
file_type            TEXT,
uploaded_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
-- Ajouts 20260213000000:
uploaded_by          UUID REFERENCES auth.users(id),
uploaded_by_type     VARCHAR(10) DEFAULT 'client'
CHECK (uploaded_by_type IN ('client', 'admin')),
is_visible_to_client BOOLEAN DEFAULT TRUE,
deleted_at           TIMESTAMPTZ,
deleted_by           UUID REFERENCES auth.users(id),
delete_reason        TEXT
);

CREATE INDEX idx_deposit_proofs_active
ON public.deposit_proofs(deposit_id) WHERE deleted_at IS NULL;
```

11.4 Référence Complète des RPCs

RPC	Acteur	Paramètres clés	Retour
create_client_deposit	Client + Admin	user_id, amount_xaf, method, bank_name, agency_name	{success, deposit_id, reference}
validate_deposit	Admin	deposit_id, admin_comment, confirmed_amount, send_notification	{success, amount_credited, old_balance, new_balance, reference}
reject_deposit	Admin	deposit_id, reason (req.), rejection_category, admin_note	{success, reference}
request_deposit_correction	Admin	deposit_id, reason (req.)	{success, reference}
start_deposit_review	Admin	deposit_id	{success}
resubmit_deposit	Client	deposit_id	{success}
cancel_client_deposit	Client	deposit_id	{success, reference, message}
get_deposit_stats	Admin	—	{total, awaiting_proof, proof_submitted, ..., to_process, today_*}

11.5 Réponses aux Questions Clés du Cahier des Charges

Q1 : Quels sont les statuts possibles et les transitions ?

Voir §3. Il existe 8 statuts : created, awaiting_proof (fantôme), proof_submitted, admin_review, pending_correction, validated, rejected, cancelled. Statuts terminaux : validated, rejected, cancelled.

Q2 : Quand le wallet client est-il crédité ?

Uniquement lors de la validation admin (validate_deposit). Le wallet n'est jamais débité dans le module dépôts. Le montant crédité = confirmed_amount si fourni, sinon amount_xaf.

Q3 : Quels boutons apparaissent dans l'écran détail selon le statut ?

Voir §6.1.4. La règle principale : isLocked = [validated, rejected, cancelled]. Si non-locked !' to bouton "Supprimer" reste pour les super_admins.

Q4 : Combien d'étapes compte le formulaire de création et quels champs ?

6 étapes (+ loader) : client !' amount !' family !' submethod (si applicable) !' bank ou agency (si a minimum sont client + montant + famille méthode. Bank/agency selon méthode.

Q5 : Quels sont les filtres disponibles sur l'écran liste ?

Voir §6.2.2. Chips de statut (6), filtres avancés (méthode + tri + période), barre de recherche libre (nom/référence/téléphone).

Q6 : Comment sont gérées les preuves ?

Voir §5. Upload client ou admin !' bucket deposit-proofs/{user_id}/{depositId}/. Soft-delete (del si aucune preuve restante. Guard de validation obligatoire (au moins 1 preuve active).

Q7 : Quels sont les problèmes actuels ?

Voir §9. 10 problèmes identifiés, les plus critiques : double écriture wallet_operations probablement cassée (§9.1), canStartReview trop restrictif (§9.3), useDeleteDeposit sans protection DB (§9.4), race condition sur l'upload (§9.6).

Q8 : En quoi ce module diffère-t-il du module paiements ?

Voir §10. Principales différences : sens du flux (crédit vs débit), pas de bénéficiaire, pas de taux de change, workflow correction/resoumission, annulation client possible, méthodes hiérarchisées (famille + sous-méthode).